COMPONENT 03 / UNIVERSITY ML PROJECT

The Complete Machine Learning Study Guide

Multimodal Disease Severity Assessment & Temporal Progression Forecasting for Cavendish Banana Plantations

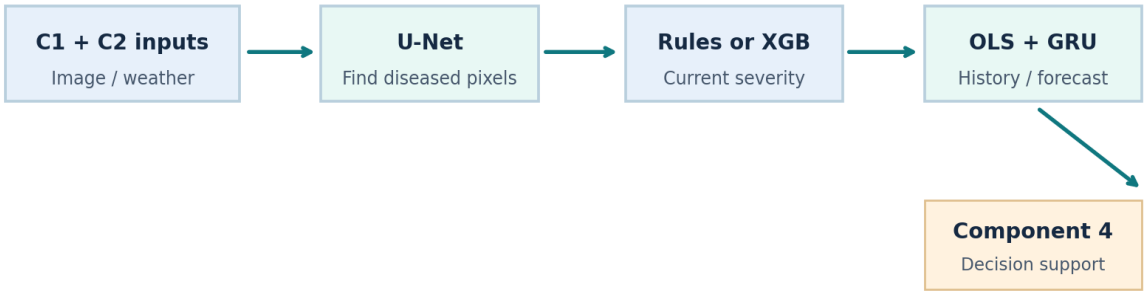


Figure 1. How Component 3 connects image evidence, severity estimation and a 7-day forecast.

HOW TO USE THIS BOOK

Read the **plain-language explanation** first, then the **technical mechanism**, then the **math and worked example**. Finish each topic by answering its viva question. This is a learning guide, not a claim that every code path was independently verified.

Grounded in: Component 3 project report (18 October 2026) and the supplied 28-page technical study guide. **Scope:** the source-code repository, datasets, checkpoints and experiment logs were not attached; their contents are described only as reported in those documents.

Prepared for learning, research comprehension, and university viva defense | October 2026

Your learning roadmap

A single idea connects the entire system: **pixels tell us where damage is; features and formulas estimate how serious it is; historical observations help us estimate what happens next.** The chapters follow exactly that sequence.

1. What Component 3 is trying to do 3

2. Exactly what data were used 5

3. U-Net: teaching a network where disease is 7

4. From segmentation pixels to biological measurements 10

5. Multimodal fusion: turning evidence into one score 12

6. XGBoost: learning severity from numerical features 14

7. How time works: SQLite, OLS and the GRU 17

8. Experiments: how to read the reported results 20

9. How the reported code fits together 23

10. A complete worked example and viva preparation 25

THREE TRAINED MODEL FAMILIES

U-Net: image → pixel mask. **XGBoost:** numerical features → severity estimate. **GRU:** historical sequence → next-week severity. A second U-Net checkpoint was trained on synthetic images.

What you are looking at	How it should be interpreted
Reported result	A metric or implementation detail stated in the submitted project report.
Worked example	Illustrative numbers selected to teach a calculation; not a new experiment.
Theory	General ML explanation that helps explain the named algorithm.
Needs verification	A claim that requires actual source code, data provenance or evaluation logs.

CHAPTER 01 / COMPONENT 3 EXPLAINED

1. What Component 3 is trying to do

Imagine two banana leaves that both have Black Sigatoka. One has a few spots; the other has large dead regions. A disease label alone cannot tell a farmer how serious these cases are. Component 3 converts a diagnosis into measurable severity and a tentative future trajectory.

AFTER THIS CHAPTER, YOU SHOULD BE ABLE TO
Explain why disease identity, current damage, environmental urgency, and future progression are different pieces of information.

1.1 The job of each component

Part	Question it answers	Information exchanged
Component 1: vision diagnosis	What disease is present?	Disease label, confidence, leaf image, plant ID.
Component 2: microclimate risk	How favorable are current weather conditions?	Zone ID, environmental urgency on a 0–100 scale.
Component 3: this project	How much damage, how serious, and what next?	Severity score/category, morphometrics, trend, 7-day estimate.
Component 4: decision support	What action might be appropriate?	Consumes Component 3 output; any treatment requires agronomic validation.

1.2 A single observation, explained

Component 3 first asks U-Net to label each image pixel as **background**, **healthy leaf**, or **lesion**. A rule-based feature calculator counts pixels and lesion regions. A disease-specific scoring method creates an **impact score**. Fusion then incorporates the environmental urgency supplied by Component 2. The result is stored so that future visits can be compared.

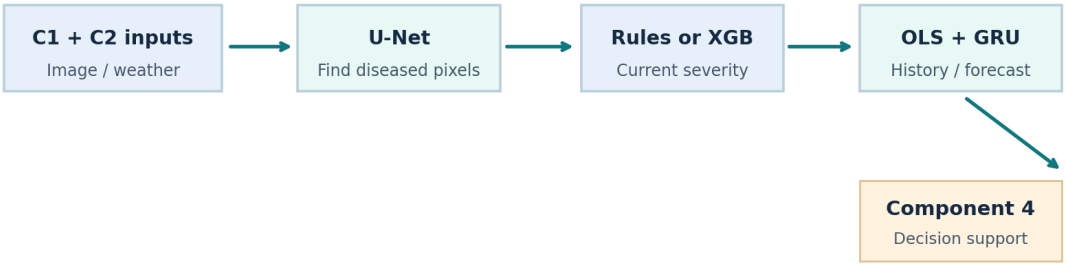


Figure 2. The major analytical stages, from external inputs to Component 4.

A useful distinction: severity is not the same as probability

A severity score of 60 does **not** mean there is a 60% chance of disease. In this project it is a constructed 0–100 index combining symptoms and environmental urgency. Similarly, “lesion scope = 18%” measures image area; it is not a 18% chance of infection.

> **ONE-SENTENCE VIVA ANSWER**
> "Component 3 quantifies visible foliar damage, combines it with disease-specific impact and climate urgency, stores inspection history, and estimates seven-day progression to support downstream decisions."

1.3 Diseases in the submitted report

Class	Disease scoring concept
Black / Yellow Sigatoka	Stover–Gauhl grade bands mapped from diseased leaf extent.
Cordana leaf spot	Horsfall–Barratt-related scope scoring plus lesion count and coalescence.
Banana bunchy top virus (BBTV)	Symptom-stage proxy and abnormal leaf shape; necrosis alone is inadequate.
Healthy	Zero biological impact in the reported baseline rule.

Disease labels are provided to Component 3 by Component 1. Segmentation helps quantify visible tissue regions; it does not independently diagnose each pathogen.

CHAPTER 02 / COMPONENT 3 EXPLAINED

2. Exactly what data were used

There are two very different learning tasks in your project. For the vision task, the computer needs images with correct pixel labels. For severity and progression, it needs numerical examples and temporal sequences. These should never be confused.

AFTER THIS CHAPTER, YOU SHOULD BE ABLE TO
Identify each training dataset, its target labels, the models trained from it, and where synthetic data limits conclusions.

2.1 Dataset-to-model map

Reported dataset	Description	Used for
Real segmentation set	1,827 real leaf photographs; 10,612 COCO annotation polygons. The report names a held-out set of 193 images.	Real-data U-Net checkpoint.
Synthetic image set	720 generated images with masks, according to the report.	Synthetic baseline U-Net checkpoint.
Synthetic longitudinal set	120 simulated plants × 6 weekly visits = 720 inspection records.	XGBoost features/targets and GRU sequence training.
Additional processed images	Report lists 3,985 processed images in five disease classes.	The exact relationship to the segmentation training set is not established by the attached materials.

DO NOT MIX THESE NUMBERS UP
1,827 counts real photographs; **10,612** counts polygons (not photographs); **720** counts simulated longitudinal records (120 plant trajectories with six visits each). The same number 720 is also reported for a synthetic image baseline, but these are conceptually distinct artifacts.

2.2 What is a COCO polygon?

An annotation is a human-supplied answer for a training example. In image segmentation the answer is not one label for the whole photograph. It is a **mask**: a pixel-by-pixel map. COCO polygons describe object boundaries using coordinate points; rasterization turns polygons into pixels. For this project, the reported mask classes are 0 = background, 1 = leaf, 2 = lesion.

1. Start with an RGB photo of a banana leaf.
2. Fill the annotated leaf polygon into class 1 on a blank mask.
3. Paint the lesion polygons over the leaf as class 2.
4. Pixels outside the annotations remain background (class 0).
5. Train the neural network to predict this reference mask from the photograph.

Image dimensions and data representation

Each resized image is 256 × 256 with three color channels (R, G, B). A PyTorch batch often has shape **[B, 3, 256, 256]**; the reference mask has shape **[B, 256, 256]**. Here B means the number of images in the batch. The model output has three logits per pixel, typically **[B, 3, 256, 256]**. These tensor shapes are the architecture implied by the report, not a verified run of its code.

2.3 Why synthesize a disease time series?

A single photograph can show the amount of disease present today; it cannot tell you how the *same plant* changed over six weeks. The report describes a generator that simulates disease severity, environmental changes and possible treatment effects for 120 plants. These simulated observations make it possible to train a sequence model before a field collection campaign exists.

> **VERY IMPORTANT RESEARCH BOUNDARY**
> The GRU can learn relationships inside the **simulator-generated** sequences. This does not show that it predicts seven-day progression correctly on real plantations. Genuine external validation requires repeated real inspections of identified plants with reliable outcome labels.

2.4 Train, validation, test: three jobs

Partition	What it should do	What not to do
Training	Adjust model parameters to reduce a loss.	Do not evaluate final performance on the same data alone.
Validation	Choose epochs, hyperparameters and model versions.	Do not treat repeated validation tuning as an untouched test.
Test	Estimate performance on held-out examples.	Do not repeatedly choose configurations using test scores.

The report states that temporal experiments use **plant-level disjoint splitting**: records for one plant are kept within one partition. That matters because six observations from the same simulated plant are strongly correlated. The full split implementation and field-image independence still require source and dataset verification.

2.5 Data preprocessing and augmentation

Preprocessing creates standardized inputs (for example resizing and normalization). **Augmentation** changes training images in plausible ways so the network learns less brittle rules. The report describes horizontal/vertical flips, brightness/contrast/saturation ColorJitter and Gaussian blur. Training image and mask geometry must stay aligned; a flipped image with an unflipped mask teaches the wrong answer.

> **CHECK YOURSELF**
> If your model sees the same plant in training and test, why might its score be too high? Because the model can exploit plant- or scene-specific patterns rather than learning transferable disease cues.

CHAPTER 03 / COMPONENT 3 EXPLAINED

3. U-Net: teaching a network where disease is

U-Net is the part that actually looks at pixel patterns. It sees an RGB image and creates a map of the image. The rules that count pixels come afterward: U-Net itself is a learned segmentation model, not a fixed formula for severity.

AFTER THIS CHAPTER, YOU SHOULD BE ABLE TO
Describe a forward pass, loss function, backpropagation, how a mask is produced, and why skip connections help.

3.1 From image to mask

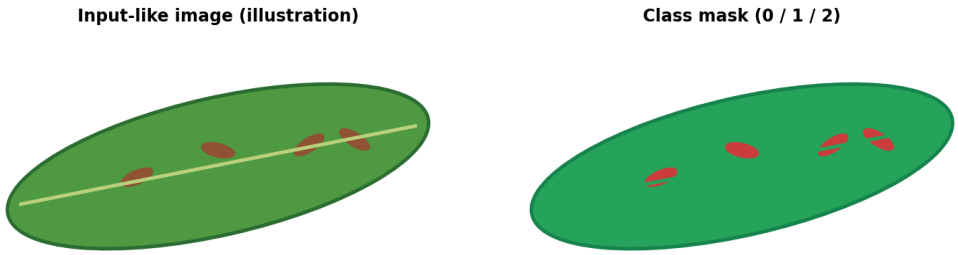


Figure 3. Illustrative banana leaf and its three-class mask. This is teaching artwork, not a sample from the real evaluation dataset.

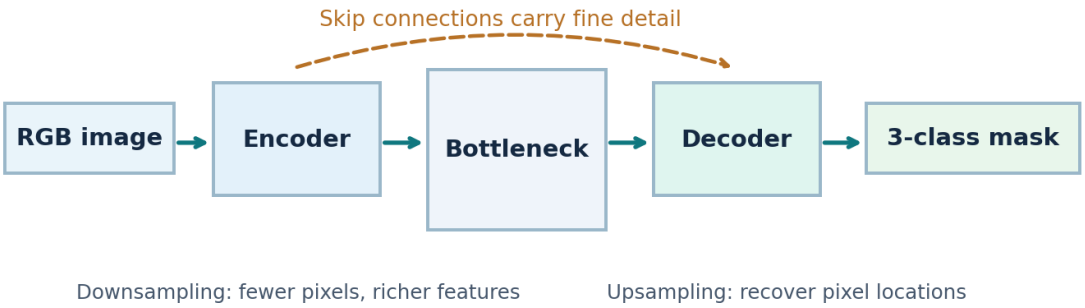


Figure 4. U-Net compresses features in the encoder and rebuilds a pixel map in the decoder; skip connections link matching resolutions.

A **convolution** slides a learned kernel (a small weight matrix) over an image or feature map. The output is a new feature map. Early filters can respond to edges and color boundaries; deeper layers combine those signals into more abstract patterns. The kernel entries are learned during training rather than programmed by hand.

Five ideas to distinguish

Term	Meaning in this U-Net
Feature map	An array of learned image responses, not a hand-written measurement.
Encoder / pooling	Progressively reduces spatial resolution while learning context.
Bottleneck	Deepest representation: broad context at low resolution.
Decoder / upsampling	Rebuilds spatial resolution for per-pixel classification.
Skip connection	Passes higher-resolution encoder detail to the corresponding decoder layer.

3.2 What is the network predicting mathematically?

The network produces three logits for each pixel: a raw score for background, leaf and lesion. **Softmax** converts them into class probabilities that sum to one. **Argmax** selects the class with the highest score. This creates the discrete 0/1/2 mask.

$$p_{i,c} = \frac{e^{z_{i,c}}}{\sum_{k=0}^{2} e^{z_{i,k}}}$$

Softmax probability for class c at pixel i, from logits z.

Example: if one pixel has logits [0.2, 1.0, 2.3], softmax gives a much larger probability to class 2. The final predicted mask labels that pixel “lesion”. Softmax is used to interpret class scores, while training loss may consume raw logits directly.

3.3 How training works: one mini-batch

1. Load a mini-batch of photographs plus their annotated masks.
2. Run the images through U-Net (**forward pass**).
3. Compute pixel-wise cross-entropy and overlap-oriented Dice loss.
4. Backpropagate the loss to calculate gradients for convolutional weights.
5. Let the optimizer update weights; repeat for more batches and epochs.
6. Use validation performance and the reported early-stopping strategy to select a checkpoint.

An **epoch** means one complete pass over the training set; a **batch** is the smaller subset used for one weight update. A **gradient** measures how the loss changes when a parameter changes. An **optimizer** uses gradients to update the parameters.

3.4 Why two losses?

$$\mathcal{L}_{total} = \mathcal{L}_{CE} + \mathcal{L}_{Dice}$$

The combined loss stated in the report.

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=0}^{2} y_{i,c}\log(p_{i,c})$$

Cross-entropy penalizes assigning low probability to the true pixel class.

$$D_c = \frac{2\sum_i p_{i,c}y_{i,c} + \varepsilon}{\sum_i p_{i,c}^2 + \sum_i y_{i,c}^2 + \varepsilon}$$

One form of differentiable soft Dice; the exact reduction/weights depend on implementation.

Cross-entropy handles classification at each pixel; Dice-oriented loss emphasizes the overlap between complete predicted and reference regions. This is useful when lesions take up relatively few pixels. **Class imbalance** means one class (e.g. background) is much more frequent than another (e.g. early lesions).

> **DO NOT SAY THIS IN YOUR VIVA**
> "U-Net calculates an 18.4% severity score." That merges two stages. A better answer is: "U-Net outputs a lesion mask; the scope calculator counts lesion pixels and divides by total leaf pixels."

3.5 What makes U-Net more effective than simple thresholds?

A color rule looks mainly at pixel color. Under changing sunlight, a yellow-green vein can resemble disease discoloration, while a real lesion may be partially shaded. U-Net can use patterns, shape, texture and spatial context, learned from annotated images. The report compares real-image segmentation on 193 held-out photographs: lesion Dice 0.7737 for U-Net versus 0.1779 for the classical baseline.

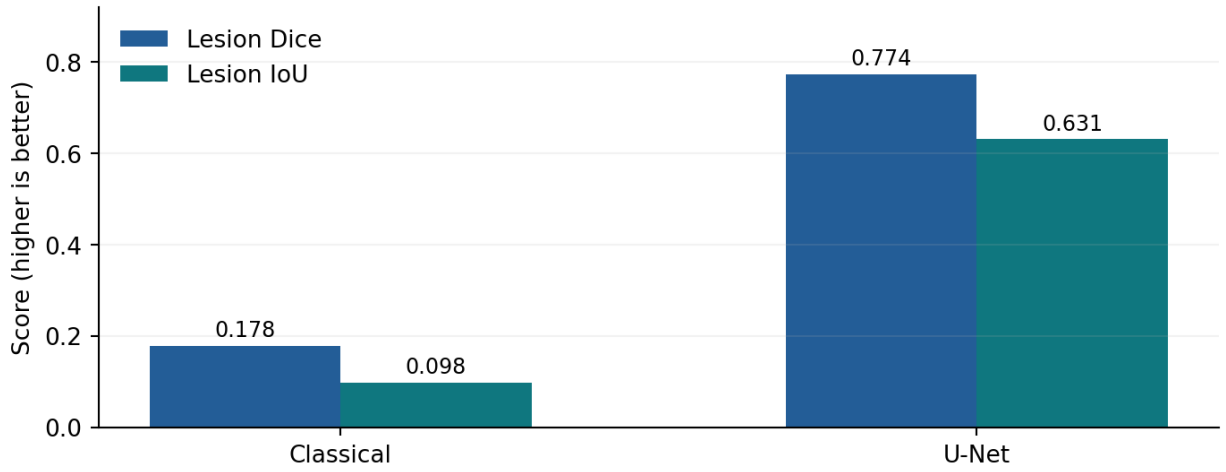


Figure 5. Reported real-image lesion overlap metrics. This is an experimental result stated in the project report, not independently re-computed here.

The increase in Dice is **0.5958 absolute, or 59.58 percentage points**. "59.58% improvement" without saying "percentage points" is potentially misleading. This is promising performance on the reported image holdout, but external farms, devices and growing conditions have not been independently tested in the provided materials.

CHAPTER 04 / COMPONENT 3 EXPLAINED

4. From segmentation pixels to biological measurements

A mask is only a grid of class labels. It becomes agriculturally useful when we turn it into features: how many pixels are damaged, how many separate lesions appear, whether lesions merge, and whether the leaf shape is unusual.

AFTER THIS CHAPTER, YOU SHOULD BE ABLE TO
Calculate scope, distinguish components from pixels, and explain why the same coverage can mean different things for different diseases.

4.1 Count the pixels correctly

Total leaf area counts **healthy leaf plus lesion pixels**, not the background. The report's worked example uses 31,351 healthy pixels and 7,069 lesion pixels.

$$S = 100 \times \frac{A_{lesion}}{A_{healthy} + A_{lesion}} = 100 \times \frac{7069}{38420} = 18.40\%$$

S is the lesion scope, a percentage of visible leaf pixels.

4.2 What are connected components?

Two nearby lesion pixels belong to the same **connected component** when there is a continuous path of lesion pixels between them under the selected connectivity definition. OpenCV can label those groups and return their areas. A count of 28 components therefore means 28 detected separated pixel regions, not necessarily 28 biologically independent infections.

Feature	Reported method	What it represents
Lesion count	Connected-component labeling	How many disconnected lesion regions appear in the mask.
Scope percentage	Lesion pixels / leaf pixels	How much visible leaf area is labeled diseased.
Coalescence ratio	Largest lesion component / total leaf area	Whether damage forms a dominant merged patch.
Aspect ratio	Leaf bounding length / width	A simple geometric proxy for unusual leaf dimensions.
Distortion index	Heuristic based on aspect ratio	A hand-designed indicator; not a clinical diagnosis of BBTV.

Worked example: coalescence

If the largest lesion is 1,421 pixels and visible leaf area is 38,420 pixels, the reported coalescence ratio is 100 × 1,421 / 38,420 = **3.70%**. If there are many small lesions but one large connected patch, count and coalescence communicate different aspects of damage.

4.3 Classical baseline: ExG, Otsu and CIELAB

Excess Green (ExG) combines color channels as 2G − R − B to emphasize green vegetation. **Otsu thresholding** selects a cutoff that separates intensity groups according to between-class variance. **CIELAB** is a perceptual color space separating lightness from color components. Their combination is a **rule-based** segmentation baseline: its rules are calculated or chosen, not learned from segmentation labels through backpropagation.

$$ExG = 2G - R - B$$

ExG is a computed color index, not a trained ML model.

> **WHERE THIS MAY FAIL**
> Color-only thresholds can confuse shadows, sun glare, chlorosis, dry leaf edges and normal veins. Conversely, U-Net can also fail when images differ substantially from its training domain; a high test Dice does not eliminate that risk.

4.4 Disease scoring is not a trained neural network

The report uses hand-specified disease-specific impact formulas. For Sigatoka, seven Stover–Gauhl grade bands map scope to an impact index. For Cordana, the score combines extent with component count and coalescence. For BBTV, an additional stunting/distortion proxy is needed because visible lesions are not a complete measure of a systemic viral infection.

Disease	Illustration of scoring logic	Caveat
Sigatoka	Map 18.40% scope into report's grade 3 band and compute I ≈ 42.33.	Band limits and interpolation are project rules requiring agronomic validation.
Cordana	Combine banded damage extent, largest-patch ratio, and lesion count.	Count and shape do not necessarily equal disease aggressiveness.
BBTV	Use reported stage proxy, scope and leaf shape distortion.	A static aspect ratio cannot itself confirm systemic BBTV stage.

CHAPTER 05 / COMPONENT 3 EXPLAINED

5. Multimodal fusion: turning evidence into one score

Fusion means combining different sources of information. In this project, three quantities are combined: visible biological impact (I), lesion scope (S) and microclimate urgency (U). The important question is whether the combination behaves sensibly in unusual conditions.

> **AFTER THIS CHAPTER, YOU SHOULD BE ABLE TO**
> Apply all three rule-based formulas, understand zero-collapse, and explain the learned XGBoost alternative.

5.1 Define I, S and U

Symbol	Meaning	Where it comes from
I	Disease-specific biological impact, on a 0–100 project index.	Impact scoring rules after morphometric extraction.
S	Percentage of visible leaf area marked as lesion.	Pixel counts in the segmentation mask.
U	Environmental urgency, also 0–100.	Provided by Component 2, not predicted by U-Net.

5.2 The three fixed formulas

Multiplicative: all factors must be nonzero

$$F_{mult} = \frac{I \times S \times U}{10000}$$

Suppose I = 42.33, S = 18.40, U = 75. Then F_mult ≈ 5.84. If urgency is zero, the entire result becomes zero even with visible disease. That is the **zero-collapse** vulnerability emphasized in the report.

Weighted sum: each signal contributes independently

$$F_{WS} = 0.45I + 0.35S + 0.20U$$

With the same example, F_WS = 0.45×42.33 + 0.35×18.40 + 0.20×75 = **40.49**. Because urgency is added rather than multiplied, even a healthy leaf could receive a nonzero fusion score if the healthy-control override is not applied; practical special-case behavior requires code verification.

Damped hybrid: an environmental multiplier

$$F_{hybrid} = (0.55I + 0.45S)\left(0.70 + 0.30\frac{U}{100}\right)$$

First calculate the biological baseline: 0.55×42.33 + 0.45×18.40 = 31.5615. Then compute the modifier: 0.70 + 0.30×(75/100) = 0.925. The final result is approximately **29.19**, which falls in the report's Moderate category.

> **IMPORTANT CORRECTION**
> The modifier ranges from **0.70 to 1.00**. It **reduces** the biological baseline when U is low and preserves all of it when U is 100. It does **not** amplify the baseline above its original value. The 70% setting is a design choice, not proof of an agronomic safety threshold.

5.3 Score categories and one surprising implication

Category	Reported range	Example
Low	0 to less than 25	21.2
Moderate	25 to less than 50	29.19
High	50 to less than 75	63
Critical	75 to 100	88

A fused severity score is a composite index, not a directly observed fraction of diseased tissue. A leaf with S = 18.4% can have a different fused score depending on I and U. The interface should display scope separately so users can distinguish physical damage from operational risk.

5.4 The optional learned fusion alternative

The report also describes an **XGBoost regressor** that ingests eight features and learns a nonlinear mapping to a synthetic target severity. The fixed formulas require no supervised training; XGBoost does. Your pipeline reportedly defaults to damped hybrid and permits XGBoost as an alternative method.

CHAPTER 06 / COMPONENT 3 EXPLAINED

6. XGBoost: learning severity from numerical features

Think of XGBoost as a committee of small decision trees. The first tree makes an initial estimate. Later trees learn how to correct the remaining mistakes. Their combined result is one predicted severity score.

AFTER THIS CHAPTER, YOU SHOULD BE ABLE TO
Identify XGBoost's inputs and target, explain sequential boosting and its objective, and interpret XGBoost versus SVR.

6.1 The reported eight inputs

Feature	Plain-language interpretation
impact_score	Disease-specific impact index from hand-designed rules.
scope_pct	Percent leaf area with lesion pixels.
urgency_score	Risk urgency supplied by Component 2.
disease_id	Numeric/categorical representation of diagnosed disease.
num_lesions	Number of distinct connected lesion regions.
coalescence_ratio_pct	Largest lesion as a share of total leaf area.
morph_distortion_idx	Leaf deformation heuristic.
leaf_aspect_ratio	Bounding length-to-width ratio.

The project reports training XGBoost on **720 simulated feature records**. An input vector is often written x R^8: eight numerical positions describing one observation. The regression target is a severity value generated within the synthetic evaluation design.

6.2 How a single decision tree predicts

A decision tree repeatedly asks threshold questions. For example: "Is biological impact above 50?" "Is the lesion scope above 20%?" Each path ends in a **leaf value**, which is a numerical prediction for observations reaching that leaf. Those thresholds are learned by minimizing a training objective, not entered as agronomic facts.

HELPFUL MENTAL PICTURE
A tree is a flowchart that learns its yes/no questions from examples. XGBoost is a sequence of these trees, where later trees correct earlier mistakes.

6.3 Why “gradient boosted”?

Let the current ensemble predict ŷ. The training loss measures the difference from the target y. A new tree is chosen to reduce the loss, using **first derivatives** (gradients) and often **second derivatives** (Hessians) of the objective with respect to the current predictions. XGBoost also penalizes unnecessarily complex trees to reduce overfitting.

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i)$$

Each new tree f_t adds a correction, scaled by learning rate η.

$$\Omega(f) = \gamma T + \frac{\lambda}{2} \sum_{j=1}^{T} w_j^2$$

Typical regularization: number of leaves T and squared leaf weights w_j.

A **gradient** shows the local direction of loss change. A **Hessian** captures the local curvature. The **learning rate** limits how much each tree changes the ensemble. **Maximum depth** controls tree complexity; **number of estimators** controls how many boosting steps are performed.

6.4 A tiny numerical example

Suppose the true synthetic training target is 40 and the current model predicts 31. An additional tree might contribute +7 after its learning-rate scaling, moving the prediction to 38. Later trees could correct smaller residual patterns. This is an **illustration of additive boosting**, not the actual tree path of your saved model.

6.5 How do Random Forest and SVR differ?

Method	How it works	Reported test RMSE ↓
Support Vector Regression (SVR)	Finds a regularized regression function using support vectors and a kernel or linear representation.	3.8678 (best of these three)
Random Forest	Fits many randomized trees and averages their predictions.	4.0203
XGBoost	Builds trees sequentially, each improving the current ensemble’s objective.	4.4526

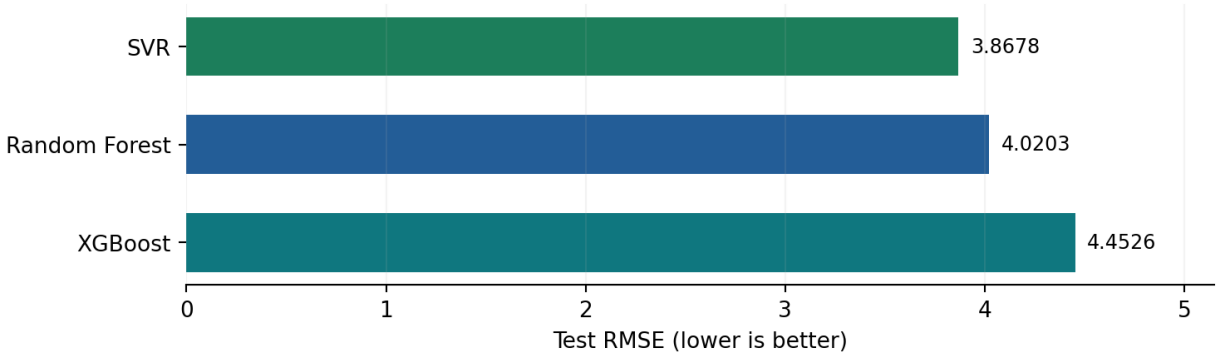


Figure 6. Reported synthetic severity-regression comparison; lower RMSE is better.

SVR, not XGBoost, has the lowest error in the reported comparison. The report favors XGBoost for engineering reasons such as feature attribution and deployment flexibility. Those are **trade-offs to test**, not proof that XGBoost

is the best predictor or inherently faster than SVR in every environment.

6.6 Feature importance: what 69.50% means

The report says impact_score represents 69.50% of XGBoost's total reported split gain and scope_pct 12.83%. Gain aggregates how much tree splits improve the training objective. It is **not** a causal statement that impact "causes 69.50% of severity." In fact, impact_score itself is calculated using some of the same disease measurements, so feature dependence must be considered.

> **A VIVA QUESTION YOU SHOULD EXPECT**
> "If SVR is more accurate, why choose XGBoost?" A defensible answer is that the reported choice favors certain engineering properties, while also acknowledging that predictive accuracy currently favors SVR and comparative deployment measurements are needed.

CHAPTER 07 / COMPONENT 3 EXPLAINED

7. How time works: SQLite, OLS and the GRU

A static severity measurement tells you how a leaf looks now; repeated measurements give a trajectory. This chapter separates three steps: store dates and scores, summarize the trend using a line, and forecast the next score with a recurrent network.

AFTER THIS CHAPTER, YOU SHOULD BE ABLE TO
Explain how weekly slopes work, why a GRU has gates and hidden state, and what happens when a plant has too little history.

7.1 SQLite is a database, not a learning model

Each new inspection is stored with a **plant identifier**, **observation date**, severity and related details. The database can later return records sorted by date for the same plant. Without stable plant IDs, the model might accidentally treat different plants as one time series.

7.2 OLS: measure whether the scores are rising

Ordinary Least Squares (OLS) fits a straight line whose predicted scores minimize the sum of squared differences from observed scores. The slope tells us how much severity changes per unit of time. Here the reported trend is expressed in severity points per seven days.

$$\beta = \frac{\sum_i (t_i - \bar{t})(s_i - \bar{s})}{\sum_i (t_i - \bar{t})^2}$$

OLS slope using inspection time t and measured severity s.

Suppose a plant has scores **16.20**, **22.40**, **29.19** on days 0, 7 and 14. The fitted slope is about **6.50 severity points per week**. Because that exceeds the report's +3 threshold, the trend is classified as **Worsening**. The threshold is a configured heuristic, not a statistical test.

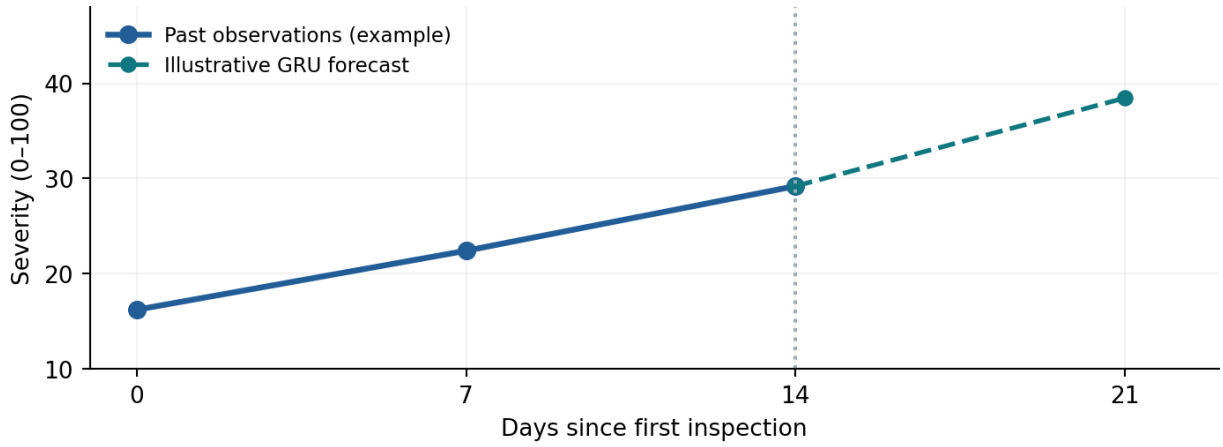


Figure 7. Illustrative historical scores and a possible next-week forecast. The 38.45 forecast is educational, not a verified checkpoint output.

7.3 What is a recurrent neural network?

A standard feed-forward network processes one input without an explicit evolving memory. A recurrent network processes a sequence x_1, x_2, ..., x and updates a **hidden state** h after every observation. The state encodes patterns from earlier visits, such as a rising slope or a recent change in trend. The recurrent weights are shared across time steps.

7.4 GRU has a reset gate and an update gate

The **update gate** controls the balance between old state and new candidate information. The **reset gate** influences how much previous information is considered when computing the new candidate. A sigmoid produces gate values between 0 and 1; tanh maps candidate activations into a bounded range.

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

Update gate; large or small z controls mixing under this stated convention.

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

Reset gate; used when constructing candidate information.

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$

Candidate state; $\odot$ means element-wise multiplication.

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Hidden state mixes past memory and newly computed candidate.

These equations illustrate a common GRU parameterization. PyTorch GRU uses an implementation-specific gate ordering and candidate calculation; source code and library calls are needed to map each symbol exactly onto saved weights.

7.5 What is trained during GRU learning?

1. The synthetic generator supplies historical severity sequences and corresponding future targets.
2. A batch groups sequences; shorter sequences may need padding to match a common length.
3. The GRU updates hidden states over the valid historical observations.
4. The last valid hidden state is projected into a predicted future severity.
5. A regression loss compares the prediction with the synthetic future target.
6. Backpropagation through time updates GRU gates and output-layer weights.

Why variable-length masking matters

Suppose one plant has three visits and another has five. A naive batch could pad the first sequence with two zeros. If the model reads **the last padded position** instead of the last actual observation, its prediction is contaminated by fake visits. The study guide describes choosing the valid hidden state by each sequence length, conceptually **output[batch_index, length − 1]**. This is reported behavior, not verified by running the source.

7.6 Cold-start logic and fallback

Number of visits	Forecast approach	Why
1	No forecast	One point gives no observed change.
2	OLS extrapolation	A slope can be computed, though extremely sensitive to noise.
3 or more	GRU prediction	The recurrent model has an actual multi-visit history.

ONE LIMITATION THAT MATTERS

A GRU can capture nonlinear structure, but it cannot manufacture real evidence. The synthetic sequences are short, and the reported trend accuracy is only 41.7%. Do not describe the forecast as field-validated.

CHAPTER 08 / COMPONENT 3 EXPLAINED

8. Experiments: how to read the reported results

An experiment should answer a specific question. Did U-Net outperform a simpler image method? Did climate help severity estimates? Did a GRU beat a naive next-week baseline? A strong researcher can explain both positive and disappointing answers.

AFTER THIS CHAPTER, YOU SHOULD BE ABLE TO
Interpret Dice, RMSE, MAE and trend accuracy, compare baselines, and explain the problem with synthetic "expert" labels.

8.1 What the metrics mean

Metric	Meaning	Better
Dice coefficient	Overlap of predicted and true lesion pixels (2TP/(2TP+FP+FN)).	Higher
IoU	Intersection divided by union of predicted and true masks.	Higher
Precision	Among predicted lesion pixels, fraction truly lesion.	Higher
Recall	Among true lesion pixels, fraction detected.	Higher
MAE	Average absolute difference between prediction and target.	Lower
RMSE	Square root of average squared prediction error; penalizes large misses.	Lower
R^2	Error relative to a mean-prediction baseline; can be negative.	Higher
Trend accuracy	Share of correctly categorized Worsening/Stable/Improving trajectories.	Higher

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

8.2 Main benchmark scoreboard

Research question	Reported comparison	Interpretation
Can real-image U-Net find lesions better than color rules?	Dice: 0.7737 U-Net vs 0.1779 classical.	Large reported advantage on the 193-image holdout.
Which severity regressor has lowest reported error?	RMSE: SVR 3.8678; RF 4.0203; XGB 4.4526.	SVR wins this particular synthetic benchmark.
Which fusion method has lowest reported RMSE?	Hybrid 3.9160; XGB 4.4526; weighted-sum 9.4704; multiplicative 29.9643.	Hybrid is best against the constructed synthetic target; note dependence.
Does GRU beat a persistence forecast?	RMSE: GRU 8.9866 vs LOCF 9.8587.	GRU improves RMSE, but not MAE or trend accuracy.

8.3 Why GRU results need careful interpretation

Forecast method	RMSE ↓	MAE ↓	Trend accuracy ↑
GRU	8.9866	6.6804	41.7%
Last observation carried forward	9.8587	6.6460	41.7%
OLS extrapolation	16.1700	11.1671	33.3%

The GRU wins **RMSE** (fewer large misses overall), while last-observation persistence wins **MAE** slightly. Both tie on trend accuracy at 41.7%, which is weak evidence for reliable direction prediction. Different metrics highlight different weaknesses; reporting only the winner is incomplete.

8.4 The subtle problem with simulated “ground truth”

The report describes targets constructed approximately as **hybrid severity plus Gaussian noise**. It calls the noisy targets independent expert ground truth. That phrase is scientifically too strong: adding random noise breaks exact numerical identity with the hybrid formula, but the target still depends directly on that formula.

$$y_{sim} = \text{clip}(F_{hybrid}(I, S, U) + \varepsilon, 0, 100)$$

This evaluation tells us how models reproduce a simulator-designed target under noise. It cannot by itself establish agreement with human agronomists or real disease progression. A proper external study would obtain blinded independent field labels and follow real plants over time.

> **WHAT YOU SHOULD SAY TO AN EXAMINER**
> “Our vision branch was reportedly tested against real polygon annotations. The severity and forecasting branches were developed and compared using simulated labels and trajectories. That enables controlled method comparisons, but true field validity remains unproven.”

8.5 Three random seeds and hyperparameter tuning

The report states seeds 42, 123 and 999 were used to repeat experiments; for XGBoost RMSE it reports 4.4268 ± 0.1997, and for GRU RMSE 8.1382 ± 1.5424. A standard deviation indicates variation across these runs; three seeds do not guarantee reproducibility or measure generalization across plantations. The report also describes a shallow XGBoost configuration with test RMSE 4.1187 while a deeper configuration was deployed for engineering reasons. Explain that trade-off honestly and ensure final model selection is based on validation rather than repeated test-set choice.

CHAPTER 09 / COMPONENT 3 EXPLAINED

9. How the reported code fits together

Your project is not one enormous network. It is a software pipeline that mixes trained ML models, fixed mathematical rules, a temporal database and an API. Learning to trace the data is more valuable than memorizing a list of filenames.

AFTER THIS CHAPTER, YOU SHOULD BE ABLE TO
Trace the important functions and data shapes, distinguish training from inference, and explain what the web API returns.

9.1 Inference: one leaf goes through these functions

Step	Reported module / function	Transformation
1	c3_orchestrator.py / _load_image()	Image path → 256×256 RGB image.
2	unet.py / UNet.predict_mask()	RGB tensor → three-class pixel mask. ClassicalSegmenter is fallback.
3	scope_calculator.py / extract_leaf_and_lesion_features()	Mask → scope, component count, coalescence and shape.
4	impact_calculators.py / calculate_disease_impact()	Disease label + morphometrics → biological impact I.
5	fusion_strategies.py / fusion_damped_hybrid()	I, S, U → severity score; XGBoost is an alternative.
6	fusion_strategies.py / map_score_to_category()	Numeric severity → Low/Moderate/High/Critical.
7	database.py / SeverityDatabase.insert_record()	Write visit; retrieve chronological plant history.
8	trend_classifier.py / calculate_ols_trend()	Timestamped scores → points/week and trend.
9	gru_forecaster.py / predict_7d()	Multi-visit sequence → next-week severity forecast, when eligible.
10	schemas/contracts.py / Component4Output	Validated structured data → Component 4.

SOURCE BOUNDARY
The preceding function and file names come from the uploaded report/study guide. The Python source itself was not supplied, so argument signatures, exact exception handling and true control flow have not been independently inspected.

9.2 Training is separate from inference

Offline training: the project loads labeled examples, updates model weights, evaluates on validation data, and saves checkpoints. **Online inference:** it loads already-trained checkpoint files and makes predictions on newly received inputs; it should not retrain after each HTTP request.

Trained artifact reported	What is stored
unet_real_weights.pt	Learned U-Net weights trained on real annotated photographs.
unet_weights.pt	Learned weights of the synthetic-image U-Net baseline.
xgboost_model.json	A learned gradient-boosted regression ensemble.
gru_forecaster.pt	GRU network weights for sequence forecasting.

9.3 A conceptual pseudocode walkthrough

The code below is **illustrative pseudocode** reconstructed from the report; it is not a copied function from the repository.

```
def assess_and_forecast(request):
    image = load_and_resize(request.image_path)
    mask = unet_predict_mask(image)
    features = measure_scope_count_shape(mask)
    impact = score_for_disease(request.disease_label, features)
    urgency = request.environmental_input.urgency_score
    score = selected_fusion(impact, features.scope_pct, urgency)
    category = categorize(score)
    store_plant_observation(request.plant_id, score)
    history = read_plant_history(request.plant_id)
    trend = ols_trend(history)
    forecast = forecast_if_enough_visits(history)
    return structured_component4_payload(score, category, trend, forecast)
```

9.4 What do Pydantic, FastAPI and SQLite contribute?

Pydantic checks input/output field types and constraints before information crosses subsystem boundaries. **FastAPI** exposes HTTP endpoints such as POST /api/v1/assess-and-forecast. **SQLite** stores inspections persistently so the next request can use historical observations. None of these tools is an ML model.

9.5 Why tests passing does not prove model validity

The report lists 35/35 automated tests passing. Functional tests can show that expected code paths execute and data contracts hold. They cannot, by themselves, prove that severity scores match independent field experts, that disease-specific thresholds are biologically optimal, or that GRU forecasts generalize beyond simulated plants.

CHAPTER 10 / COMPONENT 3 EXPLAINED

10. A complete worked example and viva preparation

This final chapter lets you rehearse the project as a coherent explanation rather than ten disconnected technologies. The same example passes through image segmentation, a severity formula, history analysis and a future prediction.

AFTER THIS CHAPTER, YOU SHOULD BE ABLE TO
Explain one complete prediction confidently, answer tough examiner questions without overclaiming, and recall essential terminology.

10.1 Follow one leaf from start to finish

Stage	Example value / calculation
Diagnosis from Component 1	Black Sigatoka; image path for a tagged plant.
Urgency from Component 2	U = 75 (given externally).
Segmentation mask	Background 27,116; healthy leaf 31,351; lesion 7,069 pixels.
Leaf scope	7,069 / (31,351 + 7,069) × 100 = 18.40%.
Disease impact	Sigatoka report's interpolated grade-3 impact I = 42.33.
Hybrid fusion	(0.55×42.33 + 0.45×18.40) × 0.925 = 29.19.
Category	Moderate because score is in [25, 50).
Persisted history	Day 0 = 16.20, day 7 = 22.40, day 14 = 29.19.
OLS trend	Approximately +6.50 points/week → Worsening.
GRU illustration	An illustrative next-week score of 38.45; not obtained by executing the checkpoint.

MEMORIZE THE RELATIONSHIP, NOT THE NUMBERS
The workflow is **pixels → mask → morphometrics → impact → fusion → stored history → trend → possible next-week forecast**. Exact sample numbers are only there to make each transformation concrete.

10.2 Tough viva questions with answers

1. Why not use image classification only?
A disease class names the condition but does not tell how much leaf tissue is affected. Segmentation supplies a spatial lesion mask for quantification.

2. What exactly was U-Net trained to predict?
Three classes per pixel: background, healthy leaf and lesion. Its target is an annotated mask, not the final 0–100 severity index.

3. Why combine cross-entropy and Dice-oriented loss?

Pixel-wise classification and region overlap are complementary objectives, especially when lesion pixels are relatively rare.

4. Why do skip connections matter?

They transfer fine spatial information from encoder to decoder so small lesion boundaries are easier to reconstruct.

5. How is leaf scope obtained?

Count predicted lesion pixels and divide by healthy-plus-lesion pixels, then multiply by 100.

6. Why disease-specific impact rules?

Different diseases present different symptom patterns; necrotic area alone may not represent BBTV stunting. Those rules still need agronomic validation.

7. Does severity 29.19 mean 29.19% damaged?

No. It is a composite index. The physical lesion scope in the example is 18.40%.

8. What is zero-collapse?

Multiplicative fusion becomes zero when any factor, including urgency, is zero—even if significant visible damage remains.

9. Does damped hybrid amplify biological severity?

No: its weather multiplier ranges from 0.70 to 1.00. It attenuates at low urgency and preserves the baseline at maximum urgency.

10. Why train XGBoost if a rule already exists?

To learn potentially nonlinear relationships among eight features. Here, however, its targets are synthetic and depend on the rule used by the simulator.

11. Why did SVR beat XGBoost in the benchmark?

The observed synthetic task may favor SVR's regression bias or hyperparameters. The scores report an outcome; they do not prove one universal explanation.

12. Is XGBoost automatically more interpretable?

It provides tree-based feature importances, but importance is not causality, and competing methods can also be analyzed with appropriate tools.

13. Why is SQLite necessary?

GRU and OLS need observations from earlier visits. Database persistence links timestamped measurements to the same plant.

14. What does an OLS slope tell you?

The average change in severity per unit of time under a fitted straight-line approximation, not a causal disease growth law.

15. What does GRU learn that OLS does not?

A GRU can fit nonlinear sequence relationships through recurrent hidden states, while OLS fits a linear trend.

16. Why is zero-padding dangerous?

Short histories padded to equal length can produce misleading final hidden states if the model treats dummy zeros as observations.

17. Why is 41.7% trend accuracy concerning?

It does not indicate dependable worsening/stable/improving discrimination. Further baselines and real field validation are needed.

18. Did the simulator create independent expert ground truth?

No. It adds noise to a hybrid-formula target, making it non-identical but still formula-dependent.

19. What evidence is strongest in this project?

The reported U-Net comparison against held-out, human-annotated real leaf images. Its exact split and label provenance should still be verified.

20. What would you test next?

Collect repeated, plant-tagged field visits with independent expert severity ratings, measure cross-farm generalization, and recalibrate scoring and uncertainty.

10.3 Essential glossary

Term	Remember this meaning
Ablation	An experiment removing one feature/group to test what it contributes.
Annotation	Human- or process-supplied reference label used as a target.
Backpropagation	Gradient calculation used to update neural network parameters.
Batch	A subset of training examples processed in one update.
Checkpoint	Saved model parameters or ensemble structure.
COCO polygon	Coordinate-based region annotation format.
Connected component	Contiguous region of mask pixels belonging to one class.
Cross-entropy	Loss penalizing incorrect class probabilities.
Data leakage	Information from evaluation examples influencing learning inappropriately.
Dice	Overlap metric or differentiable training objective for masks.
Domain shift	Difference between training data and deployment conditions.
Encoder	Network stage that compresses spatial inputs into features.
Epoch	One training pass through the examples.
Feature	Input quantity used by an ML model.
Gradient	Derivative of loss used in optimization.
GRU	Gated Recurrent Unit; recurrent model with reset/update gates.
Ground truth	Reference answer used to measure a prediction; may be real or simulated.
Hessian	Second derivative/curvature information used in XGBoost.
Hidden state	Recurrent memory-like vector summarizing previous inputs.
Hyperparameter	Configuration chosen outside the weight-learning process.
IoU	Intersection divided by union for overlap evaluation.
Lesion	Visible localized region of diseased leaf tissue.
Logit	Unnormalized class score before softmax.
Mask	Pixel grid containing class labels.
Morphometrics	Numerical geometry of leaf and lesion shapes.
OLS	Ordinary Least Squares, a linear regression fitting method.

Term	Remember this meaning
Overfitting	Learning training-specific detail that fails on new examples.
Regression	Prediction of a continuous numerical outcome.
RMSE	Root mean squared error; emphasizes large residuals.
Segmentation	Assigning a semantic class to each image pixel.
Synthetic data	Artificial examples generated by rules or simulations.
Train/validation/test	Separate data roles for fitting, choosing and assessing models.
U-Net	Encoder–decoder segmentation network with skip connections.
XGBoost	Gradient-boosted decision-tree algorithm.

10.4 A defensible two-minute project explanation

> **PRACTICE OUT LOUD**
> "Component 3 receives a diagnosed leaf image and a weather urgency score. A U-Net model trained on annotated real photographs segments background, leaf and lesion pixels. Rule-based image analysis turns that mask into measurements such as diseased area and lesion count. Disease-specific rules estimate biological impact. A damped hybrid formula, or an optional learned XGBoost regressor, combines that information into a composite severity score. The observation is stored in SQLite; OLS estimates a trend and a GRU can forecast the next week when sufficient history exists. Real-image lesion segmentation has a reported held-out benchmark, while synthetic labels and time series limit the present severity and forecast claims. Our next research step is independent longitudinal field validation."

Sources and verification status

This guide is based on the **user-supplied "Project Report — Component 3" dated 18 October 2026** and the **user-supplied 28-page "Complete Technical Study Guide"**. Algorithm explanations are standard ML theory applied to the reported design. All specific metrics, dataset counts, filenames and checkpoints are source-reported, **not independently reproduced**. No source-code ZIP, raw datasets, checkpoint bytes, exact logs, or independent agronomy papers were supplied with this request. In particular, code internals, data provenance, experimental splits, threshold calibration and field effectiveness remain unverified.